PSet 1: RNNs as models of working memory

Physics / Neuro 240: Artificial and Biological Intelligence

Format: Fill in every `# TODO` or `...` blank and answer every written-response prompt.

Submission: Upload a PDF of your completed notebook, including outputs, to Gradescope. Export the notebook as HTML, then print the HTML to PDF.

Setup

Use Python 3.12 or newer. Install the packages once, then restart the kernel if needed. The accompanying `requirements.txt` records the versions used to check this notebook.

In [2]:
```
# Uncomment this line if the packages are missing, then restart the kernel.
# %pip install -r requirements.txt

from pathlib import Path
import time
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.collections import LineCollection
from matplotlib.colors import LinearSegmentedColormap, Normalize
import jax
import jax.numpy as jnp
import optax

# Double precision helps distinguish a fixed point from a small numerical re
jax.config.update("jax_enable_x64", True)
plt.rcParams.update({"figure.figsize": (7, 4), "axes.spines.top": False,
                     "axes.spines.right": False, "font.size": 11})
COLORS = {-1: "#3566a8", 1: "#c25442"}
print("JAX", jax.__version__, "| Optax", optax.__version__)
print("Devices:", jax.devices())
```

```
JAX 0.10.0 | Optax 0.2.8
Devices: [CpuDevice(id=0)]
```

Part 1. Define the behavior · 10 points

Working memory allows information from an earlier stimulus to guide a later action. We will model this with a recurrent neural network, or RNN, whose hidden state is updated from its previous state and the current input. You will train the network on a delayed-response task, then analyze its dynamics to ask how it preserves the cue after the input

disappears.

Each trial begins with a short baseline, followed by a signed sensory cue. A negative cue means left; a positive cue means right. The cue disappears, and the network waits through a variable delay. A separate go input then tells it to respond.

Epoch	Duration in steps	Sensory input	Go input	Target output
Baseline	5	0	0	0
Cue	5	Near -1 or $+1$	0	0
Delay	20 to 50	0	0	0
Response	10	0	1	Cue sign

The output should stay near zero before the go cue. We give each response step weight 1 and each earlier step weight 0.1 in the loss. This asks the network to wait without letting the many pre-response steps dominate training. Shorter trials are padded to the maximum length; padding has loss weight zero.

Cue amplitude varies across trials and has a little sensory noise. Delay inputs are exactly zero. No input during the delay reveals the correct answer.

1A. Targets and loss weights · 5 points

Complete the two lines in the supplied generator. `respond` is a Boolean array with shape `[batch, time]`. `labels` contains one sign per trial, and `go` contains each trial's first response index. All arrays in the returned batch have the trial dimension first.

In [5]:
```
BASELINE, CUE_STEPS, RESPONSE_STEPS = 5, 5, 10
CUE_END = BASELINE + CUE_STEPS
DELAY_MIN, DELAY_MAX = 20, 50
N_INPUTS, N_UNITS = 2, 32
ALPHA = 0.2


def make_trials(rng, batch_size, delay_min=DELAY_MIN, delay_max=DELAY_MAX,
                labels=None):
    """Sample trials; use a fixed array length within each batch."""
    total_steps = CUE_END + delay_max + RESPONSE_STEPS
    if labels is None:
        labels = rng.choice([-1.0, 1.0], size=batch_size)
    labels = np.asarray(labels, dtype=float)
    assert labels.shape == (batch_size,)
    delays = rng.integers(delay_min, delay_max + 1, size=batch_size)
    go = CUE_END + delays
    t = np.arange(total_steps)[None, :]
    respond = (t >= go[:, None]) & (t < (go + RESPONSE_STEPS)[:, None])

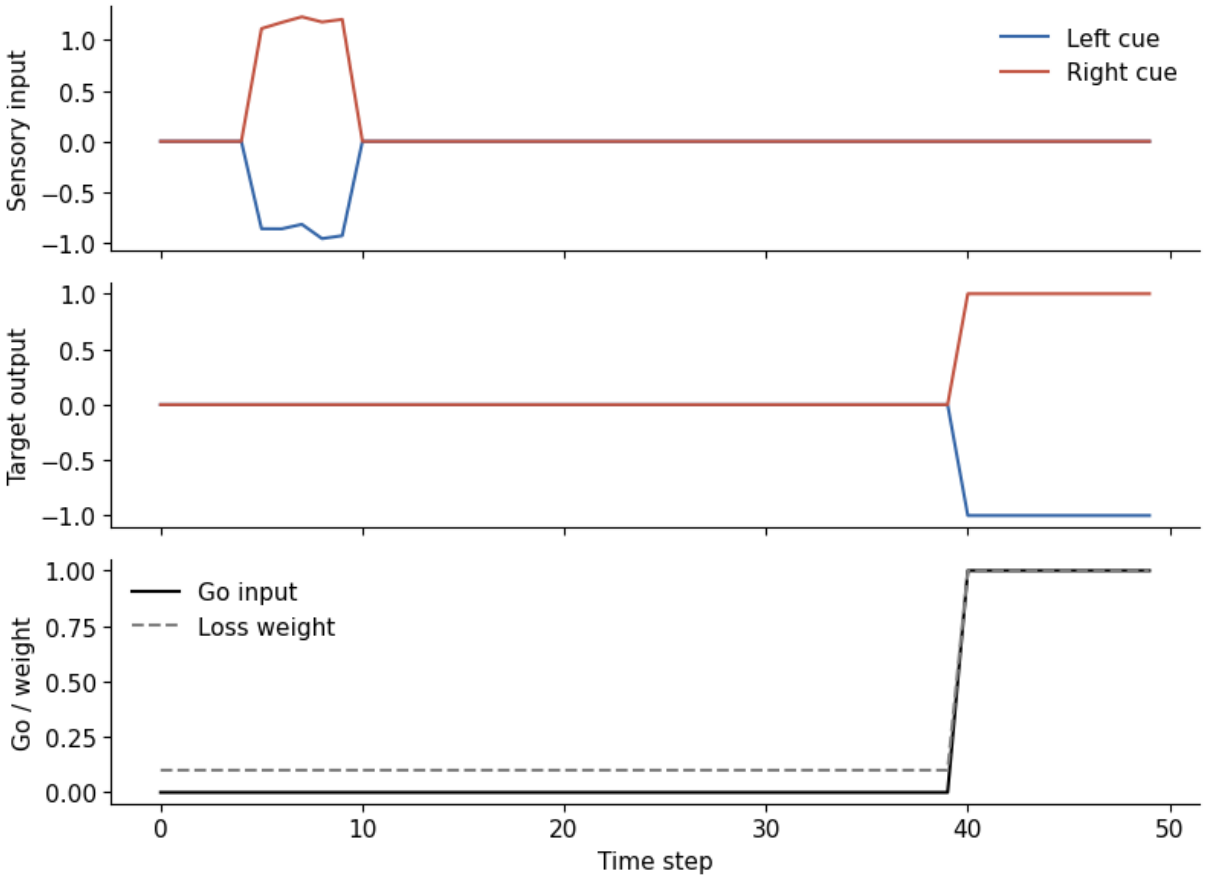
    inputs = np.zeros((batch_size, total_steps, N_INPUTS))
    amplitude = rng.uniform(0.8, 1.2, size=(batch_size, 1))
```

```
    noise = rng.normal(0, 0.05, size=(batch_size, CUE_STEPS))
    inputs[:, BASELINE:CUE_END, 0] = labels[:, None] * amplitude + noise
    inputs[:, :, 1] = respond

    # [1] The desired response is the cue sign; all other targets are zero.
    targets = respond * labels[:, None]
    # [2] Weirht pre-response steps by 0.1, response by 1, and padding by 0.
    mask = np.where(respond, 1.0, np.where(t < go[:, None], 0.1, 0.0))
    return {"inputs": jnp.asarray(inputs), "targets": jnp.asarray(targets),
            "mask": jnp.asarray(mask), "labels": labels, "go": go}


example = make_trials(np.random.default_rng(12), 2, 30, 30,
                      labels=np.array([-1.0, 1.0]))
assert example["inputs"].shape == (2, 50, 2)
np.testing.assert_allclose(example["targets"][:, :40], 0)
np.testing.assert_allclose(example["targets"][:, 40:],
                           np.repeat([[-1.0], [1.0]], RESPONSE_STEPS, axis=1
np.testing.assert_allclose(example["mask"][:, :40], 0.1)
np.testing.assert_allclose(example["mask"][:, 40:], 1.0)
# Also check the padded portion of variable-length trials.
_check = make_trials(np.random.default_rng(13), 32)
for i, go in enumerate(_check["go"]):
    assert np.all(np.asarray(_check["mask"])[i, go + RESPONSE_STEPS:] == 0)

fig, axes = plt.subplots(3, 1, figsize=(8, 6), sharex=True)
for i, label in enumerate(example["labels"]):
    axes[0].plot(example["inputs"][i, :, 0], color=COLORS[int(label)],
                 label="Left cue" if label < 0 else "Right cue")
    axes[1].plot(example["targets"][i], color=COLORS[int(label)])
axes[0].legend(frameon=False)
axes[0].set_ylabel("Sensory input")
axes[1].set_ylabel("Target output")
axes[2].plot(example["inputs"][0, :, 1], color="black", label="Go input")
axes[2].plot(example["mask"][0], "--", color="0.5", label="Loss weight")
axes[2].set(xlabel="Time step", ylabel="Go / weight")
axes[2].legend(frameon=False)
fig.tight_layout()
plt.show()
```



1B. Why recurrence? · 5 points

At the response epoch, what inputs distinguish a left trial from a right trial? Explain why a feedforward network that receives only the current input cannot solve this balanced task. What information must a successful recurrent network preserve?

Answer:

At the response epoch, the sensory input and the Go input are the same between the left and right trials (0 and 1 respectively). So, a feedforward network on the current identical input would output the same value for both trials. Since the targets are split between 1 and -1, the network would only be correct half the time. A successful recurrent network needs only to preserve the sign of the cue. Since the targets are ± 1, the amplitude of the sensory input does not matter, only the direction.

Part 2. Train a recurrent network with BPTT · 30 points

We use a leaky tanh network with a linear readout:

$$a_t = W_{\text{rec}} h_t + W_{\text{in}} u_t + b, \qquad h_{t+1} = (1-\alpha) h_t + \alpha \tanh(a_t), \qquad \hat{y}_t = w_{\text{out}}^{\mathsf{T}} h_{t+1} + b_{\text{c}}$$

Here $h_0 = 0$ and $\alpha = 0.2$. The leak is a simple way to give the state a timescale. You can view α as $\Delta t / \tau$. We analyze the discrete update actually simulated in the notebook.

Index convention: input `inputs[t]` updates h_t to h_{t+1}. The stored `states[t]` is this new state, and `outputs[t]` is its readout. Thus `states[go - 1]` is the last state before the go input arrives.

JAX treats a dictionary of arrays as a differentiable collection of parameters. Our dictionary contains `W_rec`, `W_in`, `b`, `w_out`, and `b_out`. The state-update function takes those parameters explicitly and returns a new state.

2A. One recurrent step · 6 points

Complete the preactivation and update. The shapes are `h: [units]`, `u: [inputs]`, `W_rec: [units, units]`, and `W_in: [units, inputs]`.

In [6]:
```
def init_params(seed=0):
    # Separate keys give independent draws for the three weight arrays.
    k_rec, k_in, k_out = jax.random.split(jax.random.key(seed), 3)
    return {
        "W_rec": 1.2 * jax.random.normal(k_rec, (N_UNITS, N_UNITS)) / np.sqr
        "W_in": 0.5 * jax.random.normal(k_in, (N_UNITS, N_INPUTS)),
        "b": jnp.zeros(N_UNITS),
        "w_out": jax.random.normal(k_out, (N_UNITS,)) / np.sqrt(N_UNITS),
        "b_out": jnp.array(0.0),
    }


def step(params, h, u):
    """Advance one state by one time step."""
    drive = params["W_rec"] @ h + params["W_in"] @ u + params["b"]
    h_next = (1 - ALPHA) * h + ALPHA * jnp.tanh(drive)
    return h_next


initial_params = init_params()
_zero = step(initial_params, jnp.zeros(N_UNITS), jnp.zeros(N_INPUTS))
np.testing.assert_allclose(_zero, 0)
_probe = dict(initial_params, W_rec=jnp.zeros((N_UNITS, N_UNITS)),
              W_in=jnp.zeros((N_UNITS, N_INPUTS)), b=jnp.ones(N_UNITS))
np.testing.assert_allclose(step(_probe, jnp.ones(N_UNITS), jnp.zeros(N_INPUT
                           (1 - ALPHA) + ALPHA * np.tanh(1.0))
```

2B. Unroll the recurrence · 4 points

`jax.lax.scan(body, initial_state, inputs)` runs `body` once per input step. The body returns a pair: the state to carry into the next step, and the value to save at this step. We save every new state so we can inspect the network later.

Complete the state update inside the body and the scan call. `jax.vmap` then applies this single-trial function to every trial in a batch, with the same parameters for all trials.

In [7]:
```
def rollout(params, inputs, h0):
```

```
    """Return outputs [time] and states [time, units] for one trial."""
    def body(h, u):
        h_next = step(params, h, u)
        return h_next, h_next

    _, states = jax.lax.scan(body, h0, inputs)
    outputs = states @ params["w_out"] + params["b_out"]
    return outputs, states


# None means shared; 0 means map over the first array axis.
batch_rollout = jax.jit(jax.vmap(rollout, in_axes=(None, 0, None)))
zero_state = jnp.zeros(N_UNITS)
_test_y, _test_h = batch_rollout(initial_params, example["inputs"], zero_sta
assert _test_y.shape == example["targets"].shape
assert _test_h.shape == (2, 50, N_UNITS)
np.testing.assert_allclose(_test_h[0, 0],
                           step(initial_params, zero_state, example["inputs"
```

2C. Loss and gradients · 10 points

Our loss is a weighted mean squared error across trials and time:

$$\mathcal{L}(\theta) = \frac{\sum_{i,t} m_{i,t} \left[\hat{y}_{i,t}(\theta) - y_{i,t}\right]^2}{\sum_{i,t} m_{i,t}}.$$

Complete the loss, worth 6 points, and the value-and-gradient call, worth 4 points. `jax.value_and_grad(f)` returns both the scalar function value and its gradient with respect to the first argument. Here that argument is the parameter dictionary. Differentiating through the unrolled recurrence performs backpropagation through time, or BPTT.

Optax supplies Adam and gradient clipping. The optimizer state stores running statistics used by Adam; it is separate from the network's hidden state. The update function below is compiled with `jax.jit`.

In [8]:
```
def task_loss(params, inputs, targets, mask):
    predictions, _ = batch_rollout(params, inputs, zero_state)
    loss = jnp.sum(mask * (predictions - targets)**2) / jnp.sum(mask)
    return loss


optimizer = optax.chain(optax.clip_by_global_norm(1.0), optax.adam(0.003))


@jax.jit
def train_step(params, opt_state, inputs, targets, mask):
    loss, grads = jax.value_and_grad(task_loss)(params, inputs, targets, mas
    updates, opt_state = optimizer.update(grads, opt_state, params)
    params = optax.apply_updates(params, updates)
    return params, opt_state, loss
```

```
# With zero readout, only response targets contribute to squared error.
_no_readout = dict(initial_params, w_out=jnp.zeros(N_UNITS), b_out=jnp.array
_expected = np.sum(np.asarray(example["inputs"][:, :, 1])) / np.sum(example[
np.testing.assert_allclose(task_loss(_no_readout, example["inputs"],
                                     example["targets"], example["mask"]), _e
```

2D. How does a late error reach an early state? · 10 points

For this question only, suppose the loss depends on the final state h_T. Define $J_t = \partial h_{t+1}/\partial h_t$ and the column vector $g_t = \partial \mathcal{L}/\partial h_t$.

1. Express g_t in terms of J_t and g_{t+1}. Write the resulting expression for g_0.
2. Explain how repeated contraction can make an early cue hard to learn from a late response error.
3. The same W_{rec} is used at every step. Does its gradient receive a contribution only from the final step, or from every step? Explain.

Answer:

1. $g_t = J_t^T g_{t+1}$ So, $g_0 = J_0^T J_1^T J_2^T \cdots J_{T-1}^T g_T$
2. Contraction by some factor less than 1 will reduce the size of the signal. This compounds exponentially with repetition. A late response error would be contracted many times applying the Jacobian before this signal reaches earlier time points, giving a smaller and smaller g_t as t decreases.
3. It receives a contribution from every step. Since the same weights are used across time, changes to W_{rec} ripply forward to later steps until h_T. So, the gradient is the sum of the contributions on each step and all steps contribute.

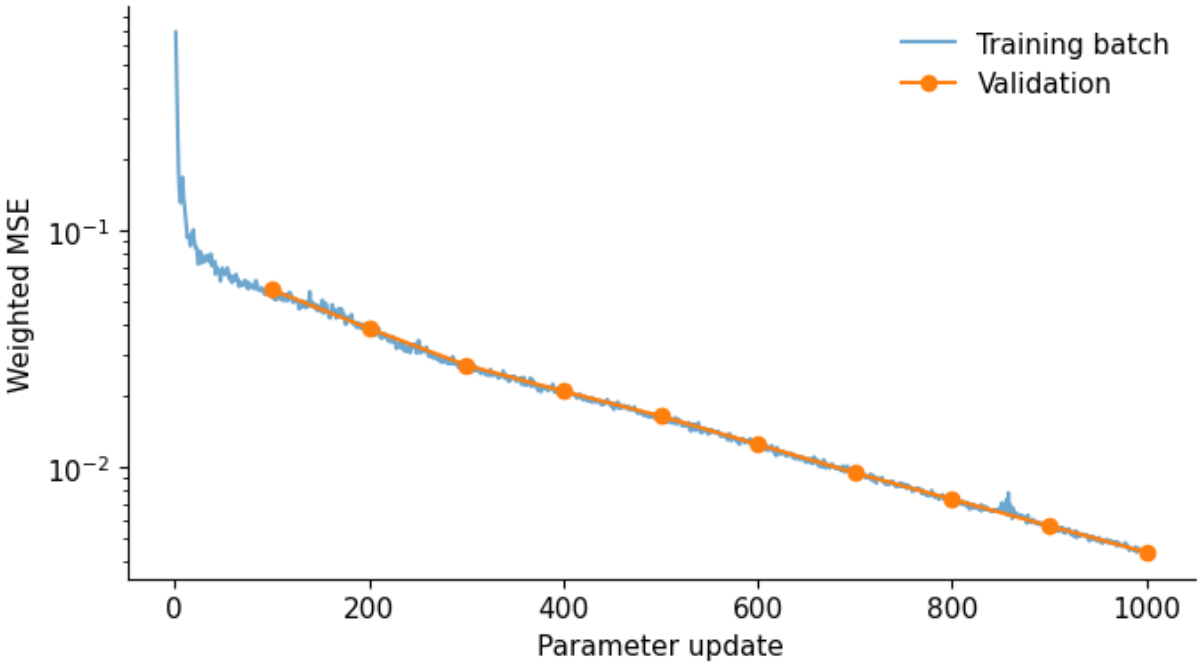
Run the supplied training loop

Train on fresh batches for 1,000 updates. The validation batch has a separate random seed and never enters a gradient update. Later behavioral checks use new test trials. Re-running this cell resets both the model and the training sampler.

In [9]:
```
TRAIN_SEED = 0
params = init_params(TRAIN_SEED)
opt_state = optimizer.init(params)
train_rng = np.random.default_rng(100)
valid = make_trials(np.random.default_rng(200), 256)
train_losses, validation_steps, validation_losses = [], [], []
start = time.perf_counter()
for iteration in range(1000):
    batch = make_trials(train_rng, 64)
    params, opt_state, loss = train_step(
        params, opt_state, batch["inputs"], batch["targets"], batch["mask"])
    train_losses.append(float(loss))
```

```
    if (iteration + 1) % 100 == 0:
        value = task_loss(params, valid["inputs"], valid["targets"], valid["
        validation_steps.append(iteration + 1)
        validation_losses.append(float(value))
print(f"Training completed in {time.perf_counter() - start:.1f} seconds.")
print(f"Final validation loss: {validation_losses[-1]:.5f}")
fig, ax = plt.subplots()
ax.semilogy(np.arange(1, 1001), train_losses, alpha=0.65, label="Training ba
ax.semilogy(validation_steps, validation_losses, "o-", label="Validation")
ax.set(xlabel="Parameter update", ylabel="Weighted MSE")
ax.legend(frameon=False)
fig.tight_layout()
plt.show()
```

```
Training completed in 2.0 seconds.
Final validation loss: 0.00440
```



Part 3. Look inside the trained network · 10 points

First check the behavior. We measure choice accuracy from the sign of the mean output during the response epoch. We also report the root mean square output before the go cue, when the target is zero. Good choice accuracy alone would not show that the network waits correctly.

The default is to analyze your trained model. If training fails or your network produces a different dynamical solution, you may set `USE_REFERENCE = True` and use the supplied checkpoint for Parts 3 through 5. State which model you used. Changing this choice requires re-running all subsequent cells.

In [10]:
```
USE_REFERENCE = False
if USE_REFERENCE:
    checkpoint = Path("rnn_reference.npz")
```

```
    if not checkpoint.exists():
        raise FileNotFoundError("Place rnn_reference.npz beside this noteboo
    with np.load(checkpoint) as saved:
        analysis_params = {name: jnp.asarray(saved[name]) for name in params
    model_name = "Supplied reference checkpoint"
else:
    analysis_params = params
    model_name = f"Your trained network, seed {TRAIN_SEED}"
print("Analyzing:", model_name)

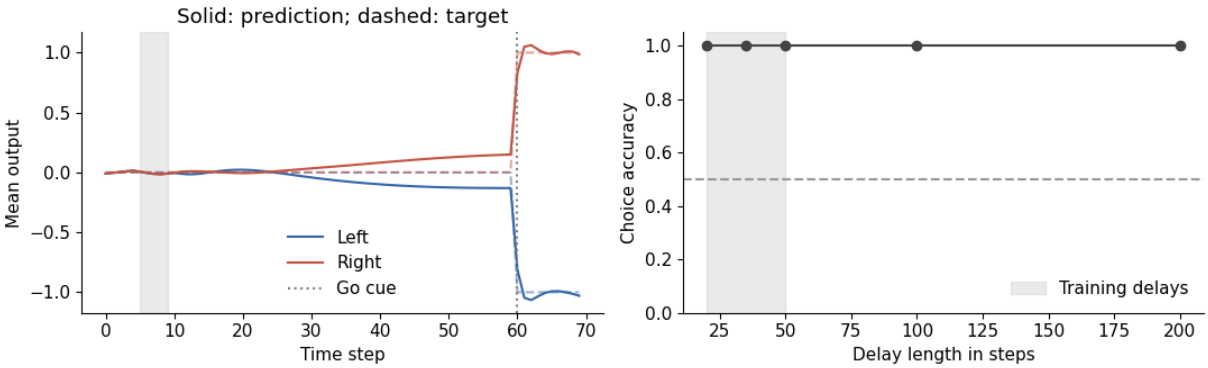

def behavior_summary(p, batch):
    outputs, states = batch_rollout(p, batch["inputs"], zero_state)
    outputs, states = np.asarray(outputs), np.asarray(states)
    response = np.asarray(batch["inputs"][:, :, 1])
    score = np.sum(outputs * response, axis=1) / np.sum(response, axis=1)
    accuracy = np.mean(np.sign(score) == batch["labels"])
    before_go = np.arange(outputs.shape[1])[None, :] < batch["go"][:, None]
    wait_rms = np.sqrt(np.mean(outputs[before_go] ** 2))
    return accuracy, wait_rms, outputs, states


# Balanced test trials at a common delay make trajectory comparisons simple.
labels = np.tile([-1.0, 1.0], 128)
test = make_trials(np.random.default_rng(300), len(labels), 50, 50, labels=l
accuracy, wait_rms, test_outputs, test_states = behavior_summary(analysis_pa
print(f"Held-out accuracy at delay 50: {accuracy:.1%}")
print(f"Pre-go output RMS: {wait_rms:.3f}")

probe_delays = [20, 35, 50, 100, 200]
delay_accuracies = []
for delay in probe_delays:
    probe = make_trials(np.random.default_rng(400 + delay), len(labels),
                        delay, delay, labels=labels)
    acc, _, _, _ = behavior_summary(analysis_params, probe)
    delay_accuracies.append(acc)

fig, axes = plt.subplots(1, 2, figsize=(11, 3.5))
for label in [-1, 1]:
    selected = test["labels"] == label
    axes[0].plot(test_outputs[selected].mean(axis=0), color=COLORS[label],
                 label="Left" if label < 0 else "Right")
    axes[0].plot(np.asarray(test["targets"])[selected][0], "--",
                 color=COLORS[label], alpha=0.5)
axes[0].axvspan(BASELINE, CUE_END - 1, color="0.8", alpha=0.4)
axes[0].axvline(test["go"][0], color="0.5", linestyle=":", label="Go cue")
axes[0].set(xlabel="Time step", ylabel="Mean output", title="Solid: predicti
axes[0].legend(frameon=False)
axes[1].plot(probe_delays, delay_accuracies, "o-", color="#444444")
axes[1].axvspan(DELAY_MIN, DELAY_MAX, color="0.8", alpha=0.4, label="Trainin
axes[1].axhline(0.5, color="0.6", linestyle="--")
axes[1].set(xlabel="Delay length in steps", ylabel="Choice accuracy", ylim=(
axes[1].legend(frameon=False)
fig.tight_layout()
plt.show()
```

```
Analyzing: Your trained network, seed 0
Held-out accuracy at delay 50: 100.0%
Pre-go output RMS: 0.076
```



A hidden state is one point in a 32-dimensional activity space. PCA gives us a two-dimensional view of that space. The code below fits one shared projection to all pre-go test states, then uses it for both cue conditions. Fixed points and perturbed states will use this same projection later.

Thin curves show eight individual trials per cue sign at half opacity. Thick curves show the mean across all trials of that sign. Blue denotes left cues and red denotes right cues; each curve darkens as the trial progresses, using the same time scale for both conditions. The black dot marks their shared starting state, circles mark cue offset, and squares mark the end of the delay.

We omit the response epoch from the PCA fit and trajectory plots because the go cue changes the inputs and therefore changes the dynamics. PCA is a view of the activity, not a fitted dynamical model.

In [11]:
```
go_index = int(test["go"][0])
pre_go_states = test_states[:, :go_index]
flat = pre_go_states.reshape(-1, N_UNITS)
pca_center = flat.mean(axis=0)
_, singular_values, vt = np.linalg.svd(flat - pca_center, full_matrices=Fals
pca_axes = vt[:2].T
explained = singular_values[:2] ** 2 / np.sum(singular_values ** 2)


def project(states):
    return (np.asarray(states) - pca_center) @ pca_axes


mean_states = {label: pre_go_states[test["labels"] == label].mean(axis=0)
               for label in [-1, 1]}


# Keep early colors visible against white; use identical time normalization.
time_norm = Normalize(vmin=0, vmax=go_index - 1)
time_cmaps = {
    label: LinearSegmentedColormap.from_list(
        name, plt.get_cmap(name)(np.linspace(0.30, 0.95, 256)))
```

```
    for label, name in [(-1, "Blues"), (1, "Reds")]
}


def add_time_trajectory(ax, states, cmap, linewidth, alpha, zorder):
    """Color each segment by its midpoint in trial time."""
    xy = project(states)
    segments = np.stack([xy[:-1], xy[1:]], axis=1)
    line = LineCollection(segments, cmap=cmap, norm=time_norm,
                          linewidths=linewidth, alpha=alpha, zorder=zorder)
    line.set_array(np.arange(len(xy) - 1) + 0.5)
    line.set_capstyle("round")
    ax.add_collection(line)
    return line

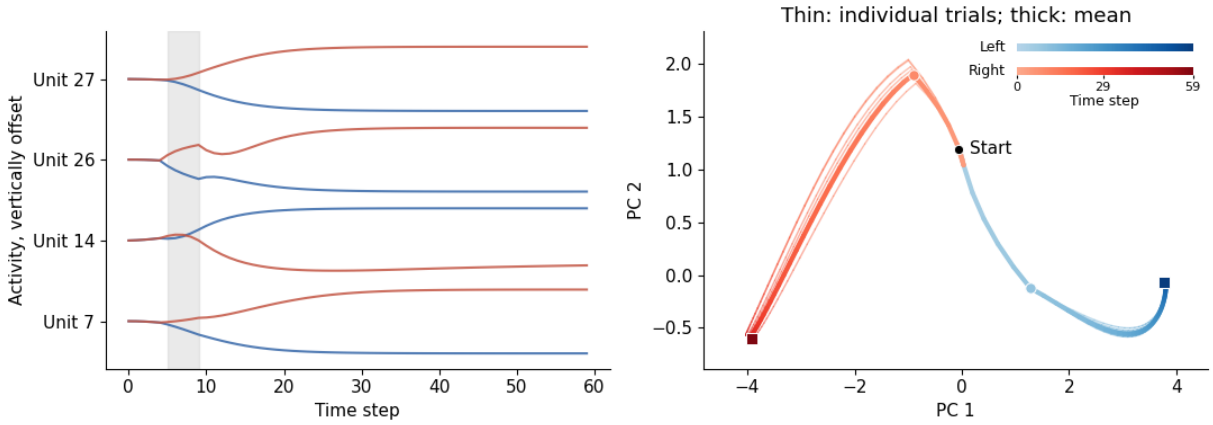

def plot_delay_trajectories(ax, n_individual=8):
    for label in [-1, 1]:
        cmap = time_cmaps[label]
        trials = pre_go_states[test["labels"] == label]
        for trial in trials[:n_individual]:
            add_time_trajectory(ax, trial, cmap, linewidth=1.0, alpha=0.5, z
        add_time_trajectory(ax, mean_states[label], cmap,
                            linewidth=3.0, alpha=1.0, zorder=3)
        trajectory = project(mean_states[label])
        # The markers use the same time colors as the lines.
        for t, marker in [(CUE_END - 1, "o"), (go_index - 1, "s")]:
            ax.scatter(*trajectory[t], marker=marker, color=cmap(time_norm(t
                       edgecolor="white", linewidth=0.7, s=45, zorder=4)

    start = project(mean_states[-1][0])
    ax.scatter(*start, s=35, color="black", edgecolor="white", linewidth=0.7
    ax.annotate("Start", start, xytext=(7, 0), textcoords="offset points", v
    ax.autoscale_view()
    ax.margins(0.10)
    ax.set(xlabel="PC 1", ylabel="PC 2")

    # Two compact keys show the same elapsed time in the two cue colors.
    for label, y, name in [(-1, 0.94, "Left"), (1, 0.87, "Right")]:
        bar_ax = ax.inset_axes([0.62, y, 0.35, 0.025])
        mappable = plt.cm.ScalarMappable(norm=time_norm, cmap=time_cmaps[lab
        bar = ax.figure.colorbar(mappable, cax=bar_ax, orientation="horizont
        bar.outline.set_visible(False)
        bar_ax.text(-0.06, 0.5, name, transform=bar_ax.transAxes,
                    ha="right", va="center", fontsize=9)
        if label == -1:
            bar.set_ticks([])
        else:
            bar.set_ticks([0, (go_index - 1) // 2, go_index - 1])
            bar.ax.tick_params(labelsize=8, length=2, pad=2)
            bar.set_label("Time step", fontsize=9, labelpad=1)


fig, axes = plt.subplots(1, 2, figsize=(11, 4))
# Choose units with large across-trial/time variance for a legible illustrat
shown_units = np.argsort(flat.var(axis=0))[-4:]
```

```
for offset, unit in enumerate(shown_units):
    for label in [-1, 1]:
        axes[0].plot(mean_states[label][:, unit] + 2.5 * offset,
                     color=COLORS[label], alpha=0.85)
axes[0].set(yticks=2.5 * np.arange(4), yticklabels=[f"Unit {u}" for u in sho
            xlabel="Time step", ylabel="Activity, vertically offset")
axes[0].axvspan(BASELINE, CUE_END - 1, color="0.8", alpha=0.4)
plot_delay_trajectories(axes[1])
axes[1].set_title("Thin: individual trials; thick: mean")
fig.tight_layout()
plt.show()
print(f"Variance shown by the first two PCs: {explained.sum():.1%}")
```




```
Variance shown by the first two PCs: 96.3%
```

3A. What do the trajectories suggest? · 10 points

Report which model you analyzed, its test accuracy, and its pre-go output RMS. Describe what happens to the two trajectories during the delay. Propose a mechanism for storing the cue, and identify one reason these plots alone cannot establish that mechanism. Does success at delays of 100 or 200 steps prove that memory lasts indefinitely?

Answer:

I used my own model. Its test accuracy was 100.0%, pre-go output RMS: 0.076. During the delay, both trajectories move down in PC2. However, the right trajectory decreases in PC1 and the left trajectory increases. To store this cue, the dynamical system behind the RNN could have two stable fixed points, one for each target. The initial state could be at an unstable fixed point, and the cue nudges it off in the corresponding direction. Without input, the dynamics of the RNN would keep the state at the target.

The trajectory map in PC space only shows where particular states went. A flow field graph that shows how other states move would help verify if stable fixed points exist.

Success at these delays does not definitively prove memory lasts indefinitely. A slow drift over a long timescale could be enough to push the state back over to the other target.

Part 4. Find and classify fixed points · 30 points

During the delay, both inputs are zero. Let $F_\theta(h, 0)$ denote one call to `step`. A fixed point h^* satisfies

$$F_\theta(h^*, 0) = h^*.$$

We search for such states by minimizing the squared update size:

$$q(h) = \frac{1}{2}\|F_\theta(h, 0) - h\|_2^2.$$

This time the **candidate state** changes and the trained weights stay fixed. This is the fixed-point-search idea used by Sussillo and Barak. It can find unstable fixed points too: stability under the original dynamics is different from being a minimum of this search objective.

A numerical minimum with a nonzero residual is not necessarily a fixed point. We will keep only candidates whose update norm is below 10^{-6} and combine duplicates. A finite search cannot prove that we found every fixed point.

4A. The search objective · 6 points

Complete `fixed_point_loss`. The code then searches from observed delay states, interpolations between the two mean delay states, and zero. Optax updates the candidate states only.

In [12]:
```
u_delay = jnp.zeros(N_INPUTS)


def fixed_point_loss(h, params, u):
    residual = step(params, h, u) - h
    return 0.5 * jnp.sum(residual **2)


# A gradient with respect to argument zero changes h, not params.
state_gradient = jax.grad(fixed_point_loss, argnums=0)
fp_optimizer = optax.adam(optax.exponential_decay(0.03, 1000, 0.3))


@jax.jit
def search_fixed_points(params, candidates):
    opt_state = fp_optimizer.init(candidates)

    def body(carry, unused):
        states, opt_state = carry
        grads = jax.vmap(state_gradient, in_axes=(0, None, None))(
            states, params, u_delay)
        updates, opt_state = fp_optimizer.update(grads, opt_state, states)
        return (optax.apply_updates(states, updates), opt_state), None

    (candidates, _), _ = jax.lax.scan(body, (candidates, opt_state), None, l
    return candidates


end_states = pre_go_states[:, -1]
left_mean = end_states[test["labels"] == -1].mean(axis=0)
right_mean = end_states[test["labels"] == 1].mean(axis=0)
fraction = np.linspace(0, 1, 33)[:, None]
interpolated = (1 - fraction) * left_mean + fraction * right_mean
# Use both cue signs; the test labels alternate left/right.
observed_seeds = end_states[:32]
candidates = jnp.asarray(np.vstack([observed_seeds, interpolated, np.zeros((
found = np.asarray(search_fixed_points(analysis_params, candidates))
objectives = jax.vmap(fixed_point_loss, in_axes=(0, None, None))(
    found, analysis_params, u_delay)
residual_norms = np.sqrt(2 * np.asarray(objectives))

RESIDUAL_TOL, DUPLICATE_TOL = 1e-6, 1e-3
unique = []
for index in np.argsort(residual_norms):
    h = found[index]
    if residual_norms[index] < RESIDUAL_TOL:
        if all(np.linalg.norm(h - previous) > DUPLICATE_TOL for previous in
```

```
            unique.append(h)
if not unique:
    raise RuntimeError("No candidates passed the residual check. Try the ref
fixed_points = np.stack(unique)
# This ordering is for readable labels, not a stability test.
ordering_direction = right_mean - left_mean
fixed_points = fixed_points[np.argsort(fixed_points @ ordering_direction)]
fp_residuals = np.array([np.sqrt(2 * float(fixed_point_loss(h, analysis_para
                        for h in fixed_points])
print(f"Accepted {np.sum(residual_norms < RESIDUAL_TOL)} of {len(found)} can
print(f"Distinct fixed points found: {len(fixed_points)}")
for i, residual in enumerate(fp_residuals):
    print(f"FP {i}: update norm {residual:.2e}")
```

```
Accepted 62 of 66 candidates.
Distinct fixed points found: 3
FP 0: update norm 1.29e-16
FP 1: update norm 1.69e-16
FP 2: update norm 2.10e-16
```

4B. Derive the state Jacobian · 8 points

Starting from the recurrent update, derive $J(h) = \partial F_\theta(h, 0)/\partial h$. Write your answer using W_{rec}, α, and the derivative of tanh. Explain why W_{rec} alone is not the state Jacobian. You may use $d\tanh(a)/da = 1 - \tanh^2(a)$.

Answer:

$$J(h) = (1-\alpha)I + \alpha \text{diag}(1 - \tanh^2(W_{\text{rec}}h + b))W_{\text{rec}}$$

4C. Compute and classify · 8 points

You may have learned that stable fixed points have Jacobian eigenvalues with negative real parts. That criterion applies to **continuous-time dynamics**. Here we take discrete steps, so the criterion is different. Both follow from asking whether a small perturbation decays.

For a continuous-time system $\dot{h} = f(h)$, linearization around a fixed point gives $\delta\dot{h} = A\,\delta h$, where $A = \partial f/\partial h$. Along an eigenmode with eigenvalue λ, its amplitude evolves as

$$c(t) = e^{\lambda t}c(0), \qquad |c(t)| = e^{\text{Re}(\lambda)t}|c(0)|.$$

It decays when $\text{Re}(\lambda) < 0$. The imaginary part produces oscillation.

For our discrete update $h_{t+1} = F(h_t)$, the Jacobian instead describes the change from one state to the next:

$$\delta h_{t+1} \approx J\,\delta h_t, \qquad J = \frac{\partial F}{\partial h}.$$

An eigenmode with eigenvalue μ is multiplied by μ at every step. After n steps,

$$c_n = \mu^n c_0, \qquad |c_n| = |\mu|^n |c_0|.$$

It decays when $|\mu| < 1$. For example, $\mu = 0.9$ retains 90% of a perturbation each step, so it is stable despite being positive. A fixed point is locally attracting if **every update eigenvalue lies inside the unit circle**. An eigenvalue outside the circle gives an unstable mode. Magnitude exactly one requires more analysis; the code marks a small neighborhood of that boundary as unresolved.

We can connect the two descriptions for this RNN. With zero delay input, define $f(h) = -h + \tanh(W_{\text{rec}} h + b)$, measuring continuous time in units of the intrinsic time constant. Our update is an Euler step $F(h) = h + \alpha f(h)$, so

$$J = I + \alpha A, \qquad \mu = 1 + \alpha\lambda.$$

With $\alpha = 0.2$, an update eigenvalue $\mu = 0.97$ corresponds to a continuous-time eigenvalue $\lambda = -0.15$. The identity term explains why the plotted eigenvalues can have positive real parts. For a finite Euler step, stability requires $|1 + \alpha\lambda| < 1$, so the step size matters too.

Read the plot below against the dashed unit circle, not against the vertical axis. It shows eigenvalues of the discrete update Jacobian J.

Complete the Jacobian call, worth 4 points, and the stability test, worth 4 points. `jax.jacrev(step, argnums=1)` differentiates with respect to `h`, the second argument. The supplied check compares autodiff against a numerical finite-difference estimate.

In [13]:
```
jacobians = np.stack([
    jax.jacrev(step, argnums=1)(analysis_params, h, u_delay)
    for h in fixed_points
])
eigenvalues = np.linalg.eigvals(jacobians)
radii = np.max(np.abs(eigenvalues), axis=1)
STABILITY_TOL = 1e-5
stable = radii < 1 - STABILITY_TOL
unstable = radii > 1 + STABILITY_TOL

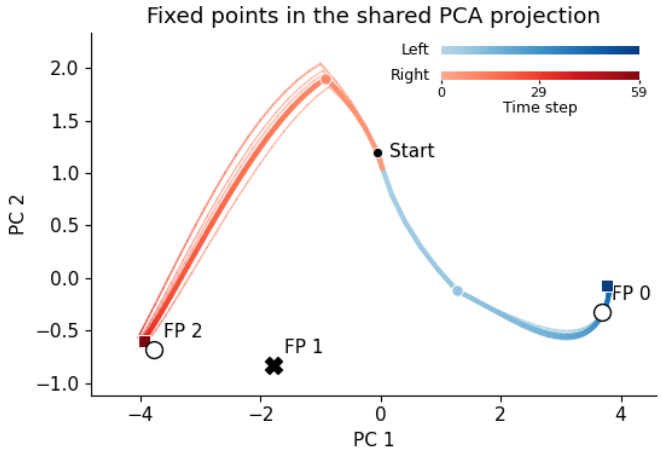
# A numerical check leaves the analytic derivation in 4B for you to do.
for h, J in zip(fixed_points, jacobians):
    delta = 1e-5
    columns = []
    for basis in np.eye(N_UNITS):
        plus = step(analysis_params, h + delta * basis, u_delay)
        minus = step(analysis_params, h - delta * basis, u_delay)
        columns.append(np.asarray(plus - minus) / (2 * delta))
    numerical = np.stack(columns, axis=1)
    np.testing.assert_allclose(J, numerical, atol=1e-8, rtol=1e-7)

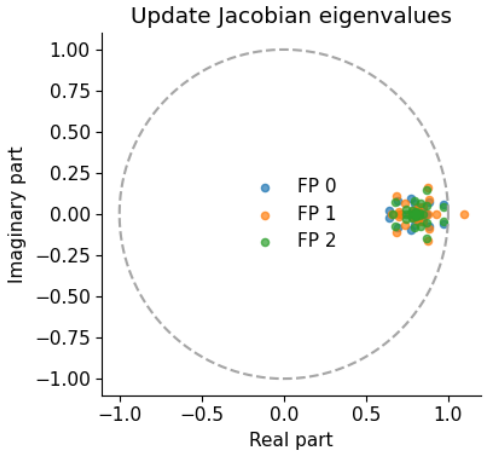
print("Point   Max |eigenvalue|   Unstable modes   Classification")
```

```python
for i, eig in enumerate(eigenvalues):
    label = "stable" if stable[i] else "unstable" if unstable[i] else "unres
    n_unstable = np.sum(np.abs(eig) > 1 + STABILITY_TOL)
    print(f"FP {i:<3}  {radii[i]:.6f}           {n_unstable:<3}

fig, axes = plt.subplots(1, 2, figsize=(11, 4))
plot_delay_trajectories(axes[0])
for i, h in enumerate(fixed_points):
    marker = "o" if stable[i] else "X"
    xy = project(h)
    axes[0].scatter(*xy, marker=marker, s=100, facecolor="white" if stable[i
                    edgecolor="black", zorder=5)
    axes[0].annotate(f"FP {i}", xy, xytext=(6, 7), textcoords="offset points
axes[0].set_title("Fixed points in the shared PCA projection")
angles = np.linspace(0, 2 * np.pi, 300)
axes[1].plot(np.cos(angles), np.sin(angles), "--", color="0.65")
for i, eig in enumerate(eigenvalues):
    axes[1].scatter(eig.real, eig.imag, s=20, alpha=0.7, label=f"FP {i}")
axes[1].set(xlabel="Real part", ylabel="Imaginary part", title="Update Jacob
axes[1].set_aspect("equal")
axes[1].legend(frameon=False)
fig.tight_layout()
plt.show()
```

```
Point   Max |eigenvalue|   Unstable modes   Classification
FP 0    0.971309           0                stable
FP 1    1.094962           1                unstable
FP 2    0.973110           0                stable
```





4D. What explains the memory? · 8 points

Report the fixed points' largest eigenvalue magnitudes and classifications. How do their locations relate to the delay trajectories? Explain why two stable states could preserve two cue identities. If you found an unstable point between them in the projection, what might it tell you, and what can you not conclude from its projected location alone?

Answer:

```
Point   Max |eigenvalue|   Unstable modes   Classification
FP 0    0.971309           0                stable
```

```
FP 1    1.094962          1                 unstable
FP 2    0.973110          0                 stable
```

FP 0 and FP 2 seem to be where the left and right trajectories go to respectively. If the system works by the mechanism I proposed earlier, the state could be nudged one way or the other by a cue. After that happens, if the dynamics push the state down to the respective fixed point, the state would likely remain in that valley and preserve the cue identities.

The unstable point FP 1 is a saddle node as it has 1 unstable mode. Its location between FP 0 and FP 2 along PC1 indicate that value of PC1 may be the boundary, left of which states would be pushed to FP 2 and right of which states would be pushed to FP 0. However, this projection doesn't show where the flow fields point around FP 1, so it may not even point toward the other fixed points.

Part 5. Perturb a memory · 18 points

An attractor account makes a prediction: a small disturbance within a memory's basin should be corrected by the dynamics. A disturbance that crosses into another basin may change the eventual choice.

We will interrupt a left-cue trial partway through its delay. Define a unit direction d pointing from the left memory fixed point toward the right one, and let D be the distance between those points. Apply

$$h \leftarrow h + \epsilon D d,$$

then continue with the original inputs and unchanged weights. Thus $\epsilon = 1$ displaces the state by the distance between the two memory points. This is a perturbation of the population state, not a new sensory cue or a weight update.

5A. Apply the perturbation · 4 points

Complete the perturbed-state line. We resume with `inputs[pulse_time:]` because `states[pulse_time - 1]` already incorporates the preceding input.

Before running the experiment, write down your prediction for a small perturbation and one comparable to the distance between memories. Use those predictions in your answer to 5B.

We sweep ϵ from 0 to 1 in steps of 0.05 to reveal trajectories between recovery and switching. The three panels use one plasma color scale for perturbation magnitude. In the trajectory panel, dashed segments show the imposed displacement and solid curves show the dynamics afterward. The gray output trace before the perturbation is shared by all conditions.

In [14]:

```
# Associate each cue with its nearest stable fixed point in the full state s
stable_indices = np.flatnonzero(stable)
if len(stable_indices) < 2:
    raise RuntimeError("This exercise needs two stable memory points. Use th
left_index = stable_indices[np.argmin(np.linalg.norm(fixed_points[stable_ind
right_index = stable_indices[np.argmin(np.linalg.norm(fixed_points[stable_in
if left_index == right_index:
    raise RuntimeError("The cues did not map to distinct stable points. Use
left_fp, right_fp = fixed_points[left_index], fixed_points[right_index]
memory_distance = np.linalg.norm(right_fp - left_fp)
direction = jnp.asarray((right_fp - left_fp) / memory_distance)

trial_index = int(np.flatnonzero(test["labels"] == -1)[0])
trial_inputs = test["inputs"][trial_index]
original_outputs = test_outputs[trial_index]
original_states = test_states[trial_index]
pulse_time = 35
assert CUE_END < pulse_time < go_index
base_state = jnp.asarray(original_states[pulse_time - 1])


def perturb_and_resume(epsilon):
    h_perturbed = base_state + epsilon * memory_distance * direction
    future_outputs, future_states = rollout(analysis_params, trial_inputs[pu
    return np.asarray(h_perturbed), np.asarray(future_outputs), np.asarray(f


# A zero perturbation must reproduce the uninterrupted trajectory.
_, continued_y, continued_h = perturb_and_resume(0.0)
np.testing.assert_allclose(continued_y, original_outputs[pulse_time:], atol=
np.testing.assert_allclose(continued_h, original_states[pulse_time:], atol=1

# A fine sweep resolves intermediate trajectories and the change in choice.
epsilons = np.round(np.linspace(0.0, 1.0, 21), 2)
# Map perturbation magnitude to the same color in every panel.
# Trim the palest yellow so large perturbations stay visible on white.
epsilon_norm = Normalize(vmin=min(epsilons), vmax=max(epsilons))
epsilon_cmap = LinearSegmentedColormap.from_list(
    "perturbation_plasma", plt.get_cmap("plasma")(np.linspace(0.05, 0.85, 25
perturbation_results = {}
fig, axes = plt.subplots(1, 3, figsize=(15, 4.8), dpi=180, layout="constrain
for ax in axes:
    ax.tick_params(labelsize=10, length=4, color="0.55")
    ax.spines["left"].set_color("0.55")
    ax.spines["bottom"].set_color("0.55")
    ax.set_axisbelow(True)

# Before the perturbation, every condition follows the same trajectory.
axes[1].plot(np.arange(pulse_time), original_outputs[:pulse_time],
             color="0.5", linewidth=2.2)
for epsilon in epsilons:
    color = epsilon_cmap(epsilon_norm(epsilon))
    h_start, future_y, future_h = perturb_and_resume(epsilon)
    perturbation_results[epsilon] = (h_start, future_y, future_h)
    # A dashed segment denotes the instantaneous jump, not recurrent evoluti
```

```
    jump = project(np.vstack([base_state, h_start]))
    if epsilon > 0:
        axes[0].plot(*jump.T, color=color, linewidth=1.5, linestyle="--",
                     alpha=0.7, zorder=1)
    delay_path = np.vstack([h_start, future_h[:go_index - pulse_time]])
    axes[0].plot(*project(delay_path).T, color=color, linewidth=2.8, zorder=
    axes[0].scatter(*project(h_start), s=26, color=color,
                    edgecolor="white", linewidth=0.6, zorder=4)
    axes[1].plot(np.arange(pulse_time, len(original_outputs)), future_y,
                 color=color, linewidth=2.7)
    distances = np.linalg.norm(future_h[:go_index - pulse_time] - left_fp, a
    axes[2].plot(np.arange(pulse_time, go_index), distances, color=color, li
    response_start = go_index - pulse_time
    response_score = future_y[response_start:response_start + RESPONSE_STEPS
    choice = "right" if response_score > 0 else "left"
    print(f"epsilon={epsilon:4.2f}: response={response_score:+.3f}, choice={

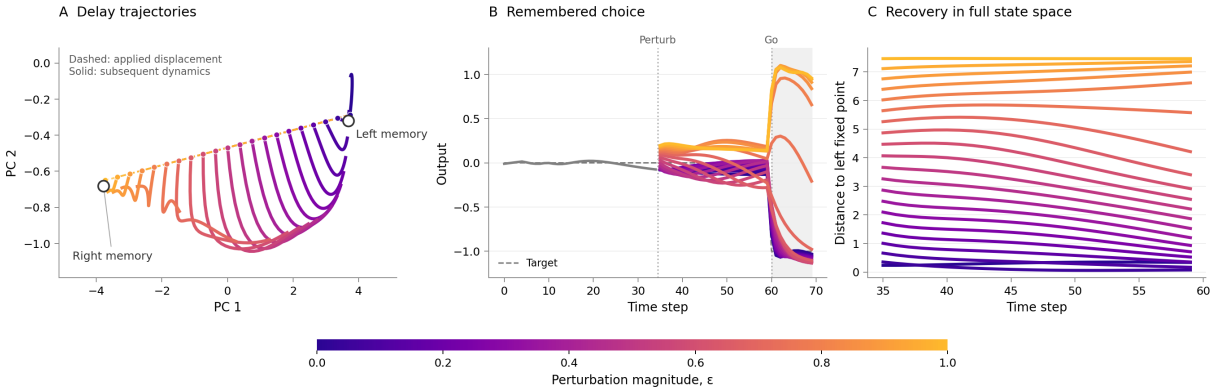
for label, h in [("Left memory", left_fp), ("Right memory", right_fp)]:
    xy = project(h)
    axes[0].scatter(*xy, s=95, facecolor="white", edgecolor="0.2", linewidth
    if label == "Left memory":
        axes[0].annotate(label, xy, xytext=(7, -15), textcoords="offset poin
                         fontsize=10, color="0.2")
    else:
        # Keep the label clear of the intermediate trajectories.
        axes[0].annotate(label, xy, xytext=(0.04, 0.08), textcoords="axes fr
                         fontsize=10, color="0.2",
                         arrowprops={"arrowstyle": "-", "color": "0.65", "lw
axes[0].set(xlabel="PC 1", ylabel="PC 2")
axes[0].set_title("A  Delay trajectories", loc="left", fontsize=12, pad=28)
axes[0].text(0.03, 0.97, "Dashed: applied displacement\nSolid: subsequent dy
             transform=axes[0].transAxes, va="top", fontsize=9, color="0.35"
axes[0].margins(x=0.18, y=0.15)

axes[1].plot(np.asarray(test["targets"])[trial_index], color="0.35",
             linestyle="--", linewidth=1.3, alpha=0.8, label="Target", zorde
axes[1].axvspan(go_index, len(original_outputs) - 1, color="0.94", zorder=0)
for t, label in [(pulse_time - 0.5, "Perturb"), (go_index, "Go")]:
    axes[1].axvline(t, color="0.65", linestyle=":", linewidth=1.2, zorder=1)
    axes[1].text(t, 1.01, label, transform=axes[1].get_xaxis_transform(),
                 ha="center", va="bottom", fontsize=9, color="0.35")
axes[1].set(xlabel="Time step", ylabel="Output", ylim=(-1.3, 1.3))
axes[1].set_title("B  Remembered choice", loc="left", fontsize=12, pad=28)
axes[1].legend(frameon=False, loc="lower left", fontsize=9)
axes[1].grid(axis="y", color="0.93", linewidth=0.7)
axes[2].set(xlabel="Time step", ylabel="Distance to left fixed point", ylim=
axes[2].set_title("C  Recovery in full state space", loc="left", fontsize=12
axes[2].grid(axis="y", color="0.93", linewidth=0.7)

# One scale applies to the curves in all three panels.
mappable = plt.cm.ScalarMappable(norm=epsilon_norm, cmap=epsilon_cmap)
colorbar = fig.colorbar(mappable, ax=axes, orientation="horizontal",
                        fraction=0.07, pad=0.10, shrink=0.55, aspect=55)
# Keep the colorbar readable even when the sweep has many samples.
colorbar.set_ticks(np.linspace(0.0, 1.0, 6))
colorbar.set_label("Perturbation magnitude, ε", fontsize=11, labelpad=5)
```

```
colorbar.ax.tick_params(labelsize=10, length=3)
colorbar.outline.set_visible(False)
plt.show()
```

```
epsilon=0.00: response=−1.000, choice=left
epsilon=0.05: response=−1.008, choice=left
epsilon=0.10: response=−1.012, choice=left
epsilon=0.15: response=−1.013, choice=left
epsilon=0.20: response=−1.013, choice=left
epsilon=0.25: response=−1.013, choice=left
epsilon=0.30: response=−1.012, choice=left
epsilon=0.35: response=−1.006, choice=left
epsilon=0.40: response=−0.993, choice=left
epsilon=0.45: response=−0.976, choice=left
epsilon=0.50: response=−0.962, choice=left
epsilon=0.55: response=−0.957, choice=left
epsilon=0.60: response=−0.955, choice=left
epsilon=0.65: response=−0.931, choice=left
epsilon=0.70: response=−0.744, choice=left
epsilon=0.75: response=+0.126, choice=right
epsilon=0.80: response=+0.842, choice=right
epsilon=0.85: response=+0.973, choice=right
epsilon=0.90: response=+1.007, choice=right
epsilon=0.95: response=+1.010, choice=right
epsilon=1.00: response=+1.006, choice=right
```



5B. Did the dynamics correct the disturbance? · 8 points

Compare your predictions with the results. Which perturbations preserve the left response, and which change it? Give the interval between adjacent sampled values where the choice switches. Use both the trajectories and the output to support your answer. Explain why local stability can predict recovery from sufficiently small perturbations but cannot determine the outcome of every large perturbation. Does being halfway between two attractors guarantee that a state lies on a basin boundary?

Answer:

The left response is preserved up until $\epsilon = 0.75$, where it flips to the right response. So, the choice switches somewhere $(0.70, 0.75)$. It switches about where I predicted, around -2 on PC1 where FP 1 is. The trajectories toward certain fixed points map well to the output; the perturbations past the 0.75 threshold lead to the output increasing at the

go signal, indicative of the rightward (+1) target.

Local stability can't determine the outcome of every large perturbation because with this, we assume linearity of the dynamics around the fixed point. However, there are two fixed points so the system cannot be linear. This nonlinearity allows for and determines the basin boundary, so linear terms can't predict when a trajectory will cross the boundary.

Being halfway does not guarantee the state lies on a basin boundary. In this case, the boundary lies closer to FP 2 than FP 0. The boundary is determined by the dynamics of the system, not just the Euclidean distance between points.

5C. RNNs as in-silico model organisms · 6 points

Researchers sometimes treat task-trained RNNs as *in-silico model organisms*. Why might this be valuable for neuroscience? Reflect on how studying a trained network's dynamics can help us describe an *algorithm* for a cognitive task, and how that description could guide experiments on the brain. Use this problem set as a starting point; you may also discuss other tasks or questions that interest you.

Then discuss the limits of this approach. What do successful task performance and an attractor mechanism in a trained RNN **not** establish about a biological circuit's implementation or learning rule?

Answer:

The brain lies either (upon oversimplification) in a wake state or a sleeping state. These states, which may be population-level activity of neurons) could be thought of as at stable fixed points that get pushed over the boundary by certain cues (amount of light, melatonin or other hormone concentration). This makes sense as people don't generally transition between sleep and wake at multiple time points in the day: we are stably at one or the other except for bedtime and wake-up.

An RNN trained to model this would highlight a basin boundary where small perturbations easily push the state to either sleep or wake. This could predict what the brain state might be like *in vivo* at the sleep-wake transition.

A model similar to the one earlier would not account for if the transition from sleep to wake occurs via a different trajectory than wake to sleep, though a different dynamical system could describe that. Though the brain's neural dynamics may be modeled by the RNN's dynamics, external factors like hormone levels, neuropeptides, and activity from neurons outside the population may change the dynamical landscape itself. This is also not modeled by our current system.

References and further reading

- Sussillo and Barak, 2013. Opening the black box: Low-dimensional dynamics in high-dimensional recurrent neural networks. The fixed-point and slow-point search approach used here. Their flip-flop example uses a different training method; this notebook borrows the analysis idea.
- Mante, Sussillo, Shenoy, and Newsome, 2013. Context-dependent computation by recurrent dynamics in prefrontal cortex. Free full text. Recommended next reading for a more complex task. Monkeys judge either the motion or the color of a stimulus, depending on context. The authors train an RNN on the same task and use fixed-point and local dynamics analysis to explain how it selects relevant evidence and integrates it toward a choice. Comparing the model with prefrontal population recordings suggests a mechanism for flexible computation in the brain, an example of the role of RNNs discussed in 5C.
- Inagaki et al., 2019. Discrete attractor dynamics underlies persistent activity in the frontal cortex. A biological connection for delayed responses and perturbation-based tests of memory dynamics.
- Barak et al., 2013. From fixed points to chaos: Three models of delayed discrimination. Different dynamics can support similar behavior.
- JAX: automatic differentiation, scan, and jacrev.
- Optax: getting started.

AI reflections · 2 points

AI assistance is allowed under the course policy. In a few sentences, describe how, if at all, you used AI while completing this problem set. What was useful? What was confusing, incorrect, or unhelpful? How did you check the suggestions or outputs you used?

If you did not use AI, say so and briefly describe how you checked your work.

You receive full credit for completing this reflection, regardless of whether you used AI or found it helpful.

Answer:

I used Claude to help generate the developer environment I ran this notebook in and insure package dependency resolution. Most of the descriptive answers mapped on to the plots generated, so I mostly just used Claude for spot-checking code syntax and math computations. Code syntax was double-checked with documentation, and computation was compared to what I had already done.